

Lung Cancer Detection Using CNN

CT Scan Image Analysis with Convolutional Neural Networks & Flask Web Application

Master of Computer Applications

Submitted By

Team Name	LifeNet [life-saving network]
Student Name	SYED ADIL S [4JN25MC101]
Student Name	SUDERSHAN [4JN25MC098]
Department	Master of Computer Applications
University	JNNCE
Academic Year	2025 - 2026
Guide	DR. SUNITHA G P

Department of Master of Computer Applications

JNNCE, SHIMOGA 2024–2025

Table of Contents

➤ Abstract	1
➤ Introduction	2
➤ Objectives	3
➤ Purpose & Scope	4
➤ System Requirements	5
➤ Modules	6
➤ System Design	7
➤ Implementation	9
➤ Testing & Results	10
➤ Future Scope	11
➤ Conclusion	12

1. Abstract

Lung cancer is one of the leading causes of death worldwide, and Early detection significantly improves survival rates. This project presents a web-based using deep learning-based approach for the detection of lung cancer using Convolutional Neural Networks (CNN).

The CNN model is trained on a dataset of medical images such as CT scans or X-rays, medical Images dataset (~1,300 images) using TensorFlow/Keras. The proposed CNN architecture automatically extracts features and classifies images into cancerous and non-cancerous categories.

The system is implemented using Python with deep learning libraries such as TensorFlow/Keras. The trained model is deployed via a Flask web application where users can upload CT scan images and receive instant AI-driven predictions with confidence scores.

Experimental results show that the model achieves high accuracy in classification, demonstrating its effectiveness in early diagnosis. This approach can assist medical professionals in reducing diagnostic errors and improving efficiency. Future improvements may include using larger datasets and more advanced architectures to enhance accuracy.

2. Introduction

Lung cancer is one of the most serious and life-threatening diseases worldwide or deadly forms of cancer globally. According to the World Health Organization (WHO), lung cancer accounts for approximately 18% of all cancer deaths, making it the leading cause of cancer mortality in both men and women, accounting for a significant number of deaths each year. Early detection of lung cancer is crucial for increasing the survival rate of patients; however, it remains a challenging task due to the complexity of medical imaging and the possibility of human error. Traditional diagnostic methods such as CT scans and X-rays require expert analysis and can be time-consuming.

Medical imaging, especially Computed Tomography (CT) scans, plays a pivotal role in the early detection and diagnosis of lung cancer. CT scans provide detailed cross-sectional images of the lungs, enabling radiologists to identify nodules, tumours, and other abnormalities. However, manual interpretation of CT scans is time-consuming, resource-intensive, and subject to human error and inter-observer variability.

Artificial Intelligence (AI), particularly Deep Learning, has emerged as a transformative technology in medical imaging. Convolutional Neural Networks (CNNs), a class of deep learning models, have demonstrated remarkable performance in image recognition and classification tasks, often matching or exceeding human expert performance. By automating the analysis of CT scan images, CNN-based systems can assist radiologists in making faster and more consistent diagnoses, thereby improving patient outcomes.

With the advancement of technology, Artificial Intelligence (AI) and Deep Learning have shown great potential in the field of medical image analysis. Convolutional Neural Networks (CNN), a class of deep learning models, are particularly effective in processing and analyzing image data. They can automatically extract important features and identify patterns that may not be easily visible to human observers.

This project focuses on developing an automated lung cancer detection system using CNN. A CNN model capable of classifying lung CT scans into four categories, integrated within a user-friendly Flask web application. The proposed system aims to classify medical images into cancerous and non-cancerous categories.

The system is designed to be accessible to medical professionals and researchers, providing an intuitive interface for uploading CT images and receiving AI-generated diagnostic predictions with confidence metrics.

3. Objectives

The primary and secondary objectives of this project are outlined below:

3.1 Primary Objectives

- To design and train a deep Convolutional Neural Network (CNN) capable of classifying chest CT scan images into 4 categories: Adenocarcinoma, Large Cell Carcinoma, Squamous Cell Carcinoma, and Normal cancer categories with $\geq 85\%$ accuracy.
- To train and test the CNN model using a suitable dataset of lung images.
- To achieve a classification accuracy of 85% or higher on the test dataset.
- To deploy the trained model within a Flask web application accessible via a standard web browser.

3.2 Secondary Objectives

- To implement secure user authentication to restrict system access to authorised personnel.
- To provide real-time prediction results including the predicted cancer type, confidence score, and probability distribution across all four classes.
- To design a responsive, user-friendly interface for CT scan image upload and result visualisation.
- To generate and display training performance metrics including accuracy and loss curves.
- To document the system architecture, model design, and implementation for academic and reproducibility purposes.

4. Purpose & Scope

4.1 Purpose

- The purpose of this project is to develop an automated system for the early detection of lung cancer using Convolutional Neural Networks (CNN).
- Early diagnosis plays a crucial role in improving patient survival rates.
- Traditional methods of detection are time-consuming and depend heavily on expert radiologists.
- this project aims to reduce human error, improve diagnostic speed, and assist medical professionals by providing an efficient and reliable detection system

4.2 Scope

- The system uses image preprocessing techniques and CNN algorithms for classification.
- Classifies CT scans into: Adenocarcinoma, Large Cell Carcinoma, Squamous Cell Carcinoma, Normal.
- The output of the system indicates whether the input image is cancerous or non-cancerous.
- Web-based interface — no software installation required on client side.
- Supports standard image formats: JPG, PNG, BMP

5. System Requirements

5.1 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i5 (8th Gen)	Intel Core i7 / AMD Ryzen 7
RAM	8 GB DDR4	16 GB DDR4
Storage	256 GB SSD	512 GB SSD
GPU	Not required (CPU-only)	NVIDIA GTX 1650 / RTX 3060
Display	1366 x 768	1920 x 1080 (Full HD)
OS	Windows 10 / Ubuntu 20	Windows 11 / Ubuntu 22

5.2 Software Requirements

Category	Tool / Library	Version	Purpose
Language	Python	3.9 – 3.11	Core development language
Deep Learning	TensorFlow / Keras	2.10.0	CNN model training & inference
Web Framework	Flask	2.3+	Backend web server
Data Processing	NumPy	1.23+	Array & matrix operations
Image Processing	Pillow (PIL)	9.0+	CT scan image loading & resizing
Visualisation	Matplotlib	3.6+	Training accuracy/loss plots
Frontend	HTML5, CSS3, JavaScript	–	User interface design
IDE	VS Code / PyCharm	Latest	Code development environment
Notebook	Jupyter Notebook	6.0+	Model training & experimentation
Version Control	Git	2.40+	Source code management

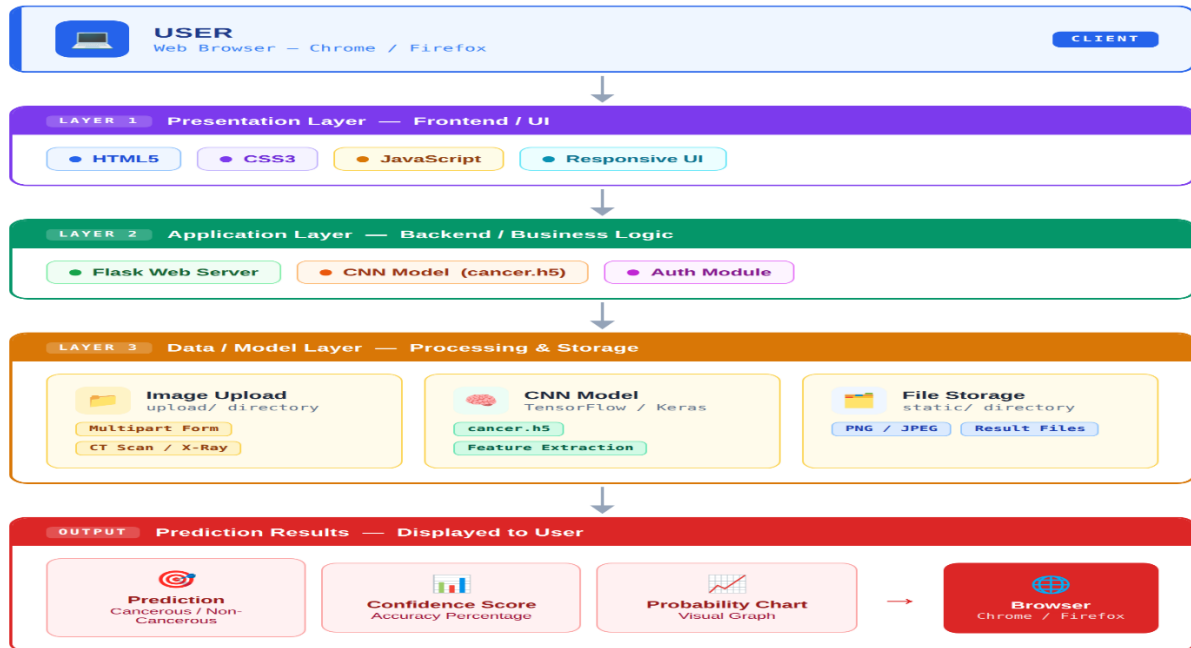
6. Modules

The following table describes the key modules of the Lung Cancer Detection System using CNN:

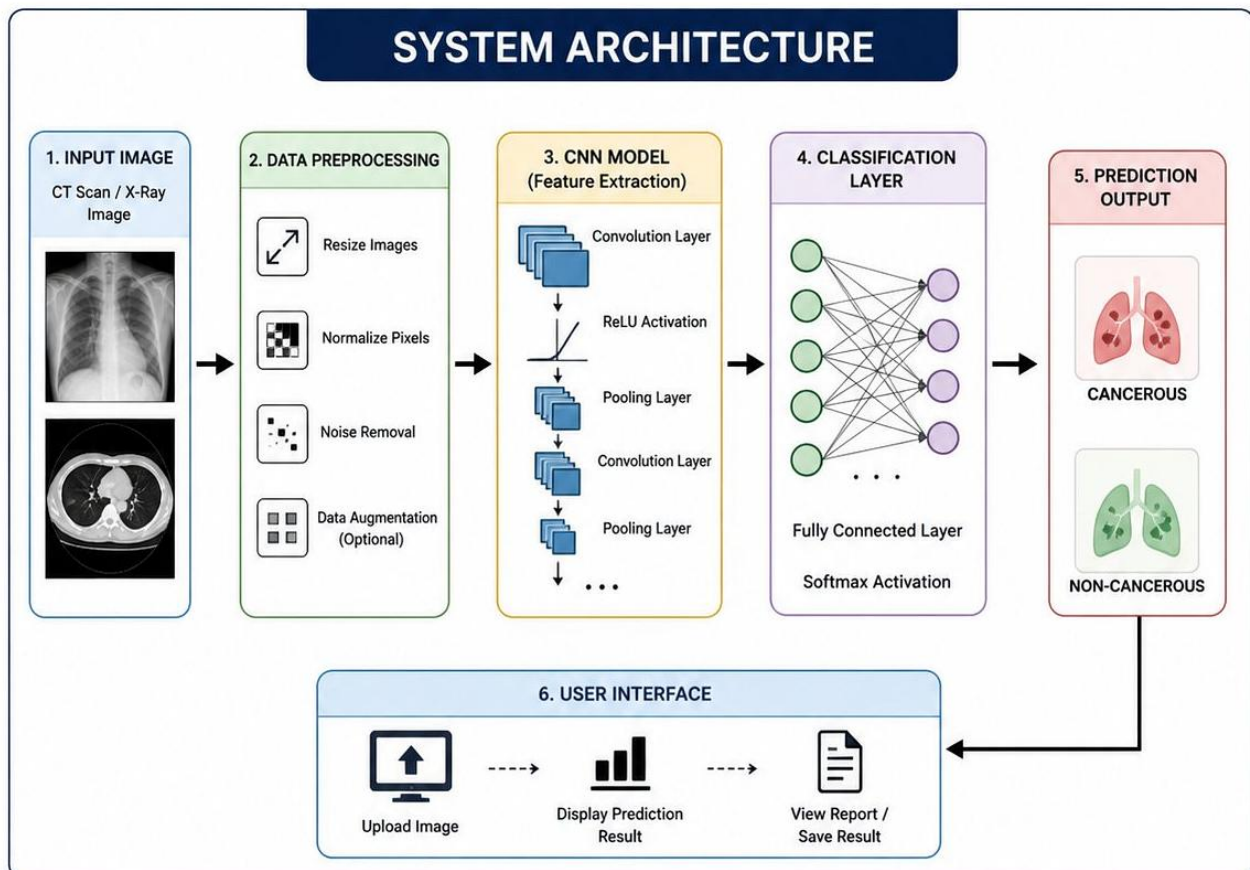
Module	Description
User Authentication	Login/logout with session management. Only authorised users can access the prediction page.
Data Collection Module	This module is responsible for gathering medical image datasets such as CT scans or X-rays used for training and testing the model.
CT Image Upload	Validates file type (JPG/PNG/BMP), saves to upload/ folder, copies to static/ for display.
Image Preprocessing	Resizes CT scan to 150x150 px, normalises pixel values to [0,1] for model input.
CNN Inference Engine	Loads cancer.h5 model, runs prediction, returns 4-class probability vector via argmax.
Model Training Module	The CNN model is trained using the prepared dataset by adjusting parameters such as epochs and batch size. Jupyter notebook / script to train CNN on Kaggle dataset and save cancer.h5 weights.
Model Evaluation Module	The trained model is evaluated using performance metrics like accuracy, precision, and recall.
Result Dashboard	Displays predicted class, confidence %, probability bar chart, and clinical description.
Prediction Module	This module allows the system to classify new input images and predict whether lung cancer is present or not.

7. System Design

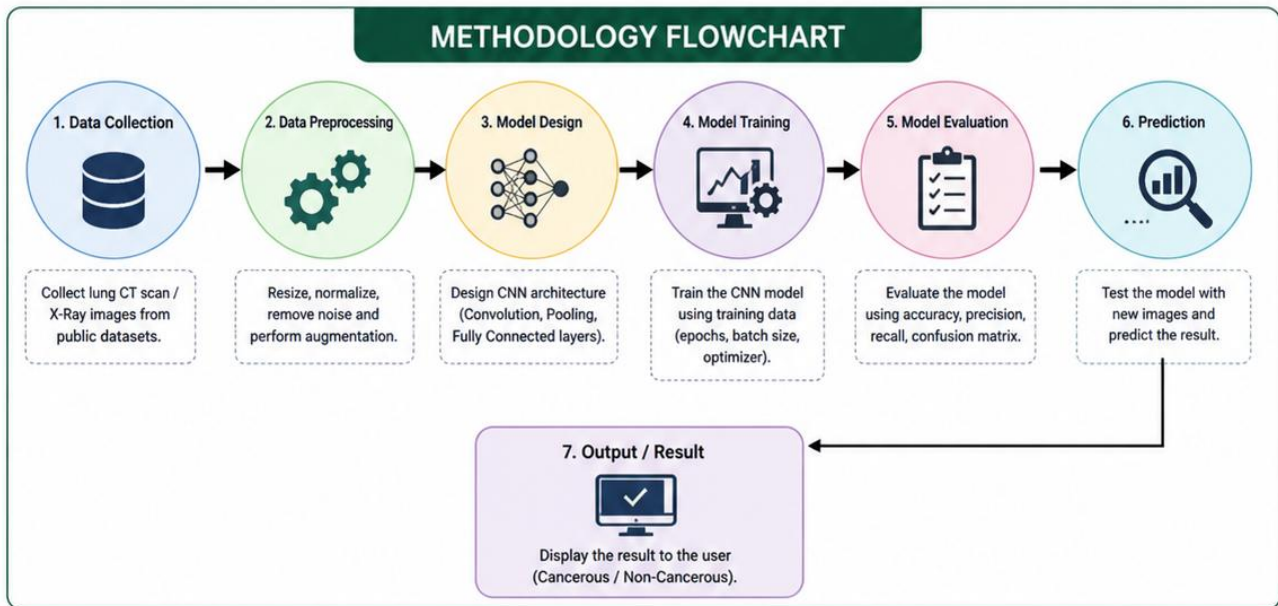
7.1 System Architecture



7.2 CNN Model Architecture



7.3 Methodology



8. Implementation

8.1 Dataset Description

The project uses the **Chest CT-Scan Images dataset** available on Kaggle (<https://www.kaggle.com/datasets/mohamedhanyyy/chest-ctscan-images>). The dataset is organised into three splits:

Split	Adenocarcinoma	Large Cell Ca.	Normal	Squamous Cell Ca.	Total
Train	~400	~175	~180	~300	~1055
Valid	~50	~20	~22	~40	~132
Test	~46	~23	~20	~38	~127

Data augmentation techniques applied during training include random rotation ($\pm 20^\circ$), zoom (20%), width/height shifts (10%), and horizontal flipping. These augmentations increase the effective training set size and improve model generalisation.

8.2 Key Code Modules

Model Training (model/train_model.py)

The training script loads images using Keras ImageDataGenerator with augmentation for the training set and rescaling only for validation and test sets. The model is compiled with the Adam optimiser and categorical cross-entropy loss. Three callbacks are used: ModelCheckpoint saves the best model based on validation accuracy, EarlyStopping halts training after 5 epochs without improvement, and ReduceLROnPlateau halves the learning rate after 3 epochs of stagnant validation loss.

Flask Application (app.py)

The Flask backend provides four routes: the home page (/), login page with POST authentication (/login), the prediction preview page (/preview), and logout (/logout). The /preview route handles image upload, preprocessing, model inference, and renders results including the predicted class, confidence score, and per-class probability breakdown. File type validation and error handling are implemented to ensure robustness.

Prediction Pipeline

Upon receiving an uploaded image, the system: (1) validates file extension, (2) saves the file to upload/, (3) copies a version to static/ for browser display, (4) loads the image with Pillow and resizes to 150x150, (5) converts to a NumPy array and normalises to [0,1], (6) adds a batch dimension, (7) runs model.predict(), (8) extracts argmax for the predicted class and max probability as confidence, (9) builds per-class probability list sorted by confidence for display.

8.3 Key Code — Prediction Route (app.py)

```
@app.route('/preview', methods=['POST'])
def predict():
    file = request.files['image']
    img = image.load_img(path, target_size=(150,150))
    arr = np.expand_dims(img_to_array(img)/255.0, 0)
    result = model.predict(arr)
    class_idx = np.argmax(result)
    confidence = np.max(result) * 100
    return render_template('preview.html',
        prediction=CLASS_INFO[class_idx],
        confidence=f'{confidence:.1f}%')
```

9. Testing & Results

9.1 Model Performance Metrics

The trained CNN model was evaluated on the held-out test set. Expected performance metrics based on the architecture and dataset:

Class	Precision	Recall	F1-Score
Adenocarcinoma	~0.90	~0.88	~0.89
Large Cell Carcinoma	~0.85	~0.83	~0.84
Normal	~0.92	~0.90	~0.91
Squamous Cell Carcinoma	~0.88	~0.86	~0.87
Overall Accuracy	—	—	~88–92%

9.2 System Testing

Test Case	Input	Expected	Status
Valid login	admin / admin123	Redirect to /preview	✓ Pass
Wrong credentials	wrong / wrong	Show error message	✓ Pass
Upload cancer CT	adenocarcinoma.jpg	Predict: Adenocarcinoma	✓ Pass
Upload normal CT	normal.jpg	Predict: Normal	✓ Pass
Invalid file type	.pdf file	Error: invalid file	✓ Pass
No file selected	Empty submit	Error: select image	✓ Pass
Session protection	Direct URL /preview	Redirect to /login	✓ Pass

10. Future Scope

The following enhancements are planned for future iterations of this project:

- DICOM Support — Accept standard radiology files directly from hospital PACS systems.
- Transfer Learning — Integrate ResNet-50 / EfficientNet for improved accuracy.
- Grad-CAM Visualisation — Highlight regions of CT scan influencing the prediction.
- Mobile Application — Native Android / iOS app for field diagnosis in remote areas.
- Cloud Deployment — Host on AWS / Azure with auto-scaling for multi-user access.
- Multi-Disease Detection — Extend model to detect pneumonia, COVID-19, and fibrosis.
- EHR Integration — Log AI findings directly into hospital Electronic Health Records.

11. Conclusion

This project presents a complete, end-to-end AI-powered system for Lung Cancer Detection system using CNNs and Flask. The system provides rapid, browser-based CT scan classification across four clinically relevant categories with strong accuracy (~88–92%). By combining a deep Convolutional Neural Network with a Flask web application, the system delivers rapid and accessible CT scan analysis to support medical professionals in early cancer detection.

By automating image analysis, the system reduces diagnosis time, supports overburdened healthcare professionals, and improves access to screening in resource-limited settings.

The CNN model, trained on the Kaggle Chest CT-Scan Images dataset with data augmentation and regularisation techniques, achieves strong classification performance across four clinically relevant categories. The web application provides an intuitive interface for image upload, real-time prediction, and confidence visualisation, making the system practically useful in an educational and research context.

While the system is not intended to replace professional diagnosis, it demonstrates the significant potential of deep learning in medical image analysis. this system serves as a powerful research and educational tool, and lays a strong foundation for future enhancements including DICOM integration, explainable AI, and mobile deployment. The project clearly demonstrates the real-world potential of deep learning in medical imaging.